

REACT CHEAT SHEET

React Topics:

Introduction , What is React? , Library vs Framework , History of React , React vs React Native , Presence of React , Components , Virtual DOM , Single Page Application , Pre-requisites , Environment Setup , Create your First React App , Create a React Component , Styles in React , Props , Conditional Rendering , Map (Dynamic Content) , Key Attribute for Components , Handling Missing Data , Sorting and Filtering , Click Events Handling , useState Hook , Methods as Props , useEffect Hook , Fetch API Data , Error Handling , Custom Hooks , onChange Event Handling , React Router , Clean Up and Updater Function , useContext Hook , useRef Hook , Dynamic Path (params) , Component Life Cycle

1. What is react

React is a component-based JavaScript library. Each part of the webpage — like heading, navbar, footer, button — is called a component. We use React to build these components and create dynamic user interfaces.

- ♦ *React is a library, not a framework.*
- **Library:** A tool we use to build components. It gives flexibility — we choose what to use and how to use it.
- **Framework:** Structure-based. It has rules and patterns we must follow while building the app.
- ♦ **Library = free style** ♦ **Framework = rule-based style**

React uses a virtual copy of the real DOM. It updates that first, compares changes, and then updates only the changed parts in the real DOM. This makes the app faster.

SPA is a type of website that loads content without reloading the whole page. It works inside a single page — no need to open new tabs or refresh the browser.

♦ **Example:** When you click a link, the content changes instantly without full reload.

2. Environment Setup

1) How to Install React JS

To install React, first download and install Node.js from nodejs.org. Node gives access to npm (Node Package Manager), which helps install React tools.

2) Setting Up React (Using Vite)

Vite is a fast frontend build tool. To create a React project using Vite:

```
npm create vite@latest
```

Choose React when prompted. Name your project (e.g., **my_app**) Navigate into your project folder :

(e.g.) – `Cd my_app npm install` Start the development server `npm run dev`

3) React Folder Structure (After Setup)

my_app — this is your main React project folder. It contains all sub-folders and files.

node_modules — contains all the libraries and dependencies used in your project.

public — holds static files like images, fonts, and icons. Anything directly accessed by the browser goes here.

src — this is the source folder where you write your React code.

src/assets — used to store images, icons, and other media files.

App.jsx — main component file. Acts like the layout or root component.

main.jsx — entry point of your React app. Connects React to the HTML file.

index.html — located inside **public**, this is the base HTML file loaded in the browser.

.gitignore — lists files/folders that should be ignored by version control (like **node_modules**).

3) 🌱 Creating a Component in React

Step 1: Inside the **src** folder, create a new file. Example: **New.jsx**

Step 2: Write the component code:

```
function New() { // 'New' is the function name (component)

  return (

    <div> // HTML elements must be wrapped inside a single div or fragment

      <h1>Hii hello</h1> // This is the content of the component

    </div>

  );

}

export default New; // Exporting the component so it can be used elsewhere
```

Step 3: To display the component in the browser, import it inside **App.jsx** (which is the default root component):

```
import New from './New'; // Importing the New component

function App() {

  return ( <>

    <New></New> // Using the New component inside App </> - wrapup element

  );

}

export default App;
```

Wrap Up (Multiple Elements)

In React, you can return only one parent element. If you want to return multiple elements, wrap them using a fragment (**<>** **</>**)

Self-Closing Tag :

If a component doesn't have children, you can use a self-closing tag: `<New />`

Accessing Variable Values

To display a value from a variable inside JSX, use curly braces `{}`:

```
const v = "hi hello";

function New() {

  return (

    <div>

      <h1>{v}</h1> // This will show "hi hello" from the variable

    </div>

  );
}
```

4. Styling in React

In React, we can style each component in 3 ways:

① Module CSS

(Mixing the `.jsx` and `.module.css` files inside a separate folder in `src`)

Step 1: Create a folder (e.g. `colour`) inside `src`. Inside that folder, create two files: `New.jsx` and `few.module.css`

`New.jsx`

```
import styles from './few.module.css'; // 'styles' is the variable

function New() {

  return (

    <div className={styles.card}> {/* 'card' is the class name */}

      <h1>Hi hello</h1>

    </div>

  );

}

export default New;
```

Few.module.css

```
.card {

  background-color: rgb(16, 129, 163); /* Styling the div */

}
```

App.jsx

```
import New from './colour/New';

function App() {

  return (

    <>

      <New />

    </>

  );

}
```

```
}  
  
export default App;
```

② External CSS

(Using a separate `.css` file like `index.css`)

New.jsx

```
import New from './colour/New';  
  
function New() {  
  
  return (  
  
    <div className="card">  
  
      <h1>Hii hello</h1>  
  
    </div>  
  
  );  
}  
  
export default New;
```



Index.css

```
.card {  
  
  background-color: rgb(116, 106, 109);  
  
}
```

③ Internal CSS

(Using a style variable inside the component)

New.jsx

```
function New() {

  const styles = {

    backgroundColor: "red"

  };

  return (

    <div style={styles}>

      <h1>Hii hello</h1>

    </div>

  );

}

export default New;
```

5. Props in React

1. What are Props?

Props (short for “properties”) are used to pass data from one component to another – usually from parent to child.

New.jsx

```
function New(props) {
  return (
    <div className="card">
      <img src={props.img}></img>
      <h1>{props.month}</h1>
      <p className='txt'>{props.desc}</p>
    </div>
  );
}
```

```
    </div>
  );
}
export default New
```

App.jsx

```
import New from './New'

import image from './assets/1.png'

import image2 from './assets/2.png'

import image3 from './assets/3.png'

function App() {

  return(

    <>

      <New img = {image} month = "January" desc = "Its a New Years New
Month"/>

      <New img = {image2} month = "February" desc = "shortest Month In
a Year" />

      <New img = {image3} month = "March" desc = "Second Odd Month"/>

    </>

  );
}

export default App;
```

2. What are Default Props?

Default props are fallback values used when a prop is not passed from the parent component.

```
New.defaultProps = {  
  
  month: "June",  
  
  desc: "Half of an year completed"  
  
};
```

React Terminology Correction:

- `props.month` → used inside the child component (**New**)
- `month="January"` → passed from the parent component (**App**)

So technically:

- **New** is the child component
- **App** is the parent component
- `month="January"` is a prop passed from parent
- `props.month` is how the child receives and uses that prop

the child is the receiver(New.jsx) and the parent is the sender(App.jsx)

6) What is Conditional Rendering in React?

Conditional rendering means showing or hiding parts of the UI based on a condition — usually a boolean (**true** or **false**) or some logic.

```
function New(props) {  
  
  if (props.display === true)  
  
  {  
  
    return (  
  
      <div className="card">  
  
        <img src={props.img} alt="month image" />  
  
      </div>  
  
    )  
  
  }  
  
}
```

```

    <h1>{props.month}</h1>

    <p className="txt">{props.desc}</p>

    <h2>{props.no}</h2>

  </div>

  );
}

else {

  return (

    <div className="card">

      <p>Month description is not available</p>

    </div>

  );
}

}

export default New; //parent class

```

App.jsx

```

function App() {

  return(

    <>

      <New img = {image} month = "January" desc = "Its a New Years New
Month" no ="1" />

```

```
    <New img = {image2} month = "February" desc = "shortest Month In  
a Year" no ="2" display= {true}/>

    <New img = {image3} month = "March" desc = "Second Odd Month" no  
="3" />

  </>

);

}

export default App;
```

“React-la condition check pannitu UI display pannrom. **props.display === true** nu check pannitu full card kaattuvom. Illa na fallback message kaattuvom. Ithu dhan conditional rendering. Ternary operator or **&&** use pannalam for short syntax.”

What Are PropTypes?

PropTypes is a runtime type-checking tool in React. It helps you validate the type of each prop passed to a component and gives warnings in the console if the type is wrong.

```
import PropTypes from "prop-types";

function New(props) {

  if (props.display === true) {

    return (

      <div className="card">

        <img src={props.img} alt="month image" />

        <h1>{props.month}</h1>

        <p className="txt">{props.desc}</p>

        <h2>{props.no}</h2>
```

```
    </div>

    );
  } else {

    return (

      <div className="card">

        <p>Month description is not available</p>

      </div>

    );
  }
}

New.propTypes = {

  month: PropTypes.string,

  desc: PropTypes.string,

  display: PropTypes.bool,

  img: PropTypes.string,

  no: PropTypes.string

};

export default New;
```



7.What is Dynamic Content Loading?

we can store all the data inside an array or list by creating an new file without being storing an duplicate data in an component file like (app.jsx)

New.jsx

```
import image from './assets/1.png';

import image2 from './assets/2.png';

import image3 from './assets/3.png';

import New from './New';

function Newlist()

{

  const elements =

  [

    {

      month: "January",

      desc: "Its a New Years New Month",

      img: image,

      no: "1"

    },

    {

      month: "February",

      desc: "Shortest Month In a Year",

      img: image2,

      no: "2"

    },

    {

      month: "March",

      desc: "Second Odd Month",
```



```
    img: image3,

    no: "3"

  }

];

const Newlist = elements.map((pick) =>

  (

    <New month={pick.month}

    desc={pick.desc}

    img={pick.img}

    no={pick.no} /> )

);

return(

  <> {Newlist}</>

);

}

export default Newlist;
```

New.jsx

```
function New(props) {

  return (

    <div className="card">

      <img src={props.img} alt="month image" />

      <h1>{props.month}</h1>

      <p className="txt">{props.desc}</p>

    </div>

  );

}
```



```

        <h2>{props.no}</h2> </div>

    );
}

export default New;

```

New.jsx

```

function New(props) {

    return (

        <div className="card">

            <img src={props.img} alt="month image" />

            <h1>{props.month}</h1>

            <p className="txt">{props.desc}</p>

            <h2>{props.no}</h2> </div>

        );
    }

    export default New;

```

App.jsx

```

import Newlist from './Newlist';

function App() {

    return (

        <>

            <Newlist/>

        </>

    );
}

```

```
}  
  
export default App;
```

Callback function

A callback function is a function that is passed as an argument to another function and is called later inside that function.

```
function greet(name, callback) {  
  console.log("Hello " + name);  
  callback(); // calling the passed function  
}
```

```
function sayBye() {  
  console.log("Bye!");  
}
```

```
greet("Dinesh", sayBye);
```

- ♦ `sayBye` is passed as a callback to `greet`
- ♦ It runs after the greeting

Arrow Function

Arrow function is a shorter way to write a function using `=>` symbol.

```
const add = (a, b) => {  
  return a + b;  
};
```

```
console.log(add(5, 3)); // Output: 8  
const add = (a, b) => {  
  return a + b;  
};
```

```
console.log(add(5, 3)); // Output: 8
```

- ♦ `=>` replaces the `function` keyword
- ♦ Cleaner and faster to write

8. Filtering And Sorting

What is Sorting?

Sorting means **rearranging data** based on a specific value — like **no**, **month**, or **price**.

in sorting we can arrange the components as per the low to high or high to low value

```
elements.sort((a, b) => a.no - b.no); // ascending
elements.sort((a, b) => b.no - a.no); // descending
```

What is Filtering?

Filtering means **selecting only specific items** from a list based on a condition.

in filter we can describe the component we can pick what component we want

```
elements.filter((item) => item.no > 1); // only items with no > 1
elements.filter((item) => item.month === "March"); // only March
```

```
import image from './assets/1.png';
import image2 from './assets/2.png';
import image3 from './assets/3.png';
import New from './New';

function Newlist() {

  const elements = [

    {
```

```

    month: "January",

    desc: "Its a New Years New Month",

    img: image,

    no: 1

  },

  {

    month: "February",

    desc: "Shortest Month In a Year",

    img: image2,

    no: 2

  },

  {

    month: "March",

    desc: "Second Odd Month",

    img: image3,

    no: 3

  }

];

// ✅ Filter items with no > 1

const filtered = elements.filter((item) => item.no > 1);

// ✅ Sort descending by no

const sorted = filtered.sort((a, b) => b.no - a.no);

```



```
//  Map to component

const Newlist = sorted.map((pick, index) => (

  <New

    key={index}

    month={pick.month}

    desc={pick.desc}

    img={pick.img}

    no={pick.no}

  />

)

);

return <>{Newlist}</>;

}

export default Newlist
```



9. Click Events in React

In React, we use the **onClick** attribute to give actions to buttons. We can trigger a function when the button is clicked.

next is click events in the click event using the `onclick()` is an attribute we can give an action to an button by creating an existing func inside the function

```
function New(props) {
  function calender(discount){ //here discount is an parameter
    console.log(props.month , "monthly calender has been " + "purchased with " + discount + " delivery charge"); //return condition
  }
}
```

```

return (
  <div className="card">
    <img src={props.img} alt="month image" />
    <h1>{props.month}</h1>
    <p className="txt">{props.desc}</p>
    <h2>{props.no}</h2>
    <button onClick={ () => calender("0rs") } >click here to buy free
monthly calender</button> //using the arrow func we can avoid repetition
of the condition in console and calender("0rs") here 0rs the argument
passed to the parameter(discount)
  </div>
);

export default New

function New(props) {
  function calender() {
    console.log(props.month , "monthly calender has been purchased"); }
  return (
    <div className="card">
      <img src={props.img} alt="month image" />
      <h1>{props.month}</h1>
      <p className="txt">{props.desc}</p>
      <h2>{props.no}</h2>
      <button onClick={calender}>click here to buy free monthly
calender</button>
    </div>
  );
}

export default New;

```

10.Hooks in React

— hooks — usestate hooks

What is a Hook in React?

A Hook is a special function in React that lets you “hook into” React features like state, lifecycle, or context — inside a function component.

What is `useState` Hook?

`useState` is a React Hook that lets you create a state variable inside a function component. You can read and update that value, and React will re-render the UI when it changes.

Syntax :

```
const [value, setValue] = useState(initialValue);
```

Here the

`purchased` → current value (initially false)

`setpurchased` → function to update the value

`useState(false)` → hook with default value `false`

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0); // initial value is 0

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  );
}

export default Counter;

//count la value store pannirukkom. setCount(count + 1) kudukkumbothu,
React adha paathu screen update pannum. Ithu dhan useState hook power."
-----
import { useState } from "react";

function New(props) {
  const [purchased, setPurchased] = useState(false); // initial state = false

  function calender() {
    console.log(props.month, "monthly calender has been purchased");
  }
}
```

```

    setPurchased(true); // update state to true
    console.log(purchased); // still shows old value (false) due to async
  update
  }

  return (
    <div className="card">
      <img src={props.img} alt="month image" />
      <h1>{props.month}</h1>
      <p className="txt">{props.desc}</p>
      <h2>{props.no}</h2>
      <button onClick={calender}>click here to buy free monthly
calender</button>
      <p>{purchased ? "already getted" : "get it now"}</p>
    </div>
  );
}

export default New;

```

`useState(false)`

Creates a state variable `purchased` with initial value `false`

`setPurchased(true)`

Updates the value to `true` when button is clicked

Re-render

React re-renders the component when state changes

Conditional UI

`purchased ? "already getted" : "get it now"` shows message based on state

Console log issue

`console.log(purchased)` shows old value because state updates are async

11.Method-as-Props in React

In normal props, we send data (like strings, numbers, images) from parent to child.

In method-as-props, we send functions from parent to child – so child can trigger actions defined in parent.

Normal props → You send data from parent to child 🧠 Example: name, image, description, etc.

“Ithu static info maadhiri — child just displays it.”

Method as props → You send a function from parent to child 🧠 Example: `handleClick`, `onBuy`, `onDelete`

“Ithu action maadhiri — child can trigger parent logic.”

Key diff : “ Normal props-la data pass pannuvom. Method as props-la function pass pannuvom. Child-ku power kudukkura maadhiri — parent logic run panna permission kudukkrom. ”

```

Counter.jsx (Child Component)
function Counter(props) {
  return (
    <div>
      /* Button click aagumbothu, parent-la irukkura function run aagum */
      /* "Dinesh" nu data pass pannrom to parent function */
      <button onClick={() => props.cli("Dinesh")}>CLICK</button>
    </div>
  );
}
export default Counter; // Counter component export pannrom

-----

App.jsx(parent class)

import Counter from './Counter'; // Counter component import pannrom

function App() {
  // Parent-la oru function define pannrom
  function click(name) {
    // Button click aagumbothu, alert message display pannum
  }
}

```

```

    alert("Hi " + name + ", welcome to learning React");
  }

  return (
    <>
      { /* Counter component-ku click function pass pannrom as props */}
      <Counter cli={click} />
    </>
  );
}
export default App; // App component export pannrom

"Parent-la function define pannuvom. Adha child-ku props-la anuppuvom.
Child-la button click pannumbothu, parent function run aagum. Ithu dhan
method as props."

```

12. useEffect Hook in React

useEffect is a method (hook) that contains a function. It is used to notice changes in a component and run code when something updates.

```

syntax :
useEffect(() => {
  // lines of code
}, [dependency]);

```

- Dependencies are the state variables or props that **useEffect** watches.
- If any value inside the dependency array changes, **useEffect** will run again.

```

import { useEffect, useState } from "react";

function New(props) {
  const [purchased, setPurchased] = useState(false); // initial state = false
  const [dummybawaah, setdummybawaah] = useState(false); // another state

  // ✅ FIX 1: useEffect should have a dependency array
  useEffect(() => {
    console.log("here comes the useeffects");
  }, []);
}

```

```

    setdummybawaah(true); // ✓ FIX 2: use correct function name
  (setdummybawaah)
  }, []); // empty array → run only once after first render

// ✓ Button click function
function calender() {
  console.log(props.month, "monthly calender has been purchased");
  setPurchased(true); // update state to true
}

return (
  <div className="card">
    <img src={props.img} alt="month image" />
    <h1>{props.month}</h1>
    <p className="txt">{props.desc}</p>
    <h2>{props.no}</h2>

    {/* ✓ Button to trigger calendar purchase */}
    <button onClick={calender}>
      click here to buy free monthly calender
    </button>

    {/* ✓ Conditional rendering based on purchased state */}
    <p>{purchased ? "already getted" : "get it now"}</p>

    {/* ✓ FIX 3: onClick should be a function, not direct call */}
    <button onClick={() => setdummybawaah(false)}>click man</button>
  </div>
);
}
export default New;

```

useEffect Dependency Types — Tech Definitions :

1 No Dependency

```

useEffect(() => {
  // runs after every render
});

```

Runs after every render (initial + updates)

Used for tasks that need to track all changes

② Empty Dependency

```
useEffect(() => {
  // runs only once (on mount)
}, []);
```

Runs only once when the component mounts

Ideal for one-time setup (e.g., API call, timer)

③ With Dependency [state]

```
useEffect(() => {
  // runs only when 'state' changes
}, [state]);
```

- **Runs only when specified state or prop changes**
- Used for reacting to specific updates (e.g., form input, toggle)

Promise in JavaScript

A **Promise** is a special JavaScript object used to handle **asynchronous operations** – like API calls, timers, or file loading.

It represents a **value that may be available now, later, or never.**

```
const promise = new Promise((resolve, reject) => {
  // async task
});
```

resolve() → called when the task is successful

reject() → called when the task fails

```
const myPromise = new Promise((resolve, reject) => {
  let success = true;
  if (success) {
```

```

    resolve("Task completed");

  } else {

    reject("Task failed");

  }

});

myPromise

  .then((result) => console.log(result))    // if resolved

  .catch((error) => console.log(error));    // if rejected

resolve() → promise fulfilled → goes to .then()

reject() → promise failed → goes to .catch()

result and error are just variable names – you can rename them

.then() --- Runs when the Promise is resolved (success)

.catch() --- Runs when the Promise is rejected (failure)

result --- Value returned by resolve() inside the Promise

error --- Value returned by reject() inside the Promise

```

13.Fetch Api Data In React

fetch() in React

Used to get data from a **server** (online or local)-("Server-la irukkura data screen-ku bring pannradhu dhan fetch.")

API Meaning

API means that creating an server and storing the data and accessing it("React-ku backend data venumna, API bridge use pannudhu.")

syntax of fetch in react

```

usestate( () =>

    fetch("url") //http://localhost:3000/elements

    .then(response => {

        (console.log(response)

    }) )

```

There is an free api data provider web is json placeholder

And to access our own data we have to create an new folder

and inside it we have to create an file we have to name the file at end with .json

```

{

    "elements": [

        {

            "Month": "January",

            "Description": "Its a New Years New month",

            "Img": "/assets/1.png",

            "Number": 1

        }

    ]

}

```

and to run the command on terminal to get the local host Command :

```
npx json-server --watch data/fakedata.json --port 3000 --static ./data
```

and after we create this it will give as an local host server and by accessing it we can fetch the data

1] Imports & State

```
-----

import { useEffect, useState } from "react";

useState → component-ku state create panna
useEffect → side effects (data fetch, console logs)

const [purchased, setPurchased] = useState(false);
const [dummybawaah, setdummybawaah] = useState(true);
const [error, setError] = useState(null);
const [loading, setLoading] = useState(true);

purchased → user calendar purchase panniyachuna state
dummybawaah → temporary toggle button state
error → fetch la error vandha message store pannum
loading → fetch nadakkumbodhu true → fallback display pannum
```

2] useEffect → Fetch Data

```
-----

useEffect(() => {

  const timer = setTimeout(() => {

    fetch("http://localhost:3000/elements") // example

    .then(response => {

      if (!response.ok) throw new Error("Could not retrieve data");

      return response.json();

    })

  }, 1000)

})
```

```

    })

    .then(data => {

        console.log("Fetched data:", data);

        setLoading(false);

    })

    .catch(err => {

        console.log("Fetch error:", err.message);

        setError(err.message);

        setLoading(false);

    });

}, 1000);

}, []);

```

Step by Step

useEffect(..., []) → only component first render la run aagum

setTimeout 1 second delay → simulate network delay

fetch() → API call panna try pannum

if (!response.ok) → HTTP error → throw error → catch block run

.then(data) → fetch success → console log → loading=false

catch(err) → fetch fail → console log → error=err.message →
loading=false

Key: Ippo error or loading state store aagum, so UI la display panna ready.

3] Functions & State Updates



```

function calender() {

  console.log(props.month + " monthly calendar has been purchased");

  setPurchased(true);

}

```

User button click → purchased=true → component re-render

UI update: "Already getted" show aagum

```

useEffect(() => { console.log(purchased); }, [purchased]);

useEffect(() => { console.log(dummybawaah); }, [dummybawaah]);

```

Any state change → console la log aagum

Debug purpose

4] Conditional Rendering (if blocks)

```

if (error) {

  return <p>{error}</p>;

}

if (loading) {

  return <p>Loading...</p>;

}

```

Logic

if (error) → fetch fail aana UI la only error show → component exit aagum

e.g., endpoint wrong → "Could not retrieve data" UI la varum

if (loading) → fetch nadakkumbodhu → "Loading..." UI la show → component exit aagum

fetch success aana apram → normal UI render

♦ Important: if block return → component immediately exit aagum. Adhukkaga loading/error only display once.

5) Normal Component UI

```
return (
  <div className="card">
    <img src={props.img}/>
    <h1>{props.month}</h1>
    <p className="txt">{props.desc}</p>
    <h2>{props.no}</h2>
    <button onClick={calender}>
      Click here to buy free monthly calendar
    </button>
    <p>{purchased ? "Already getted" : "Get it now"}</p>
    <button onClick={() => setdummybawaah(false)}>
      Click man
    </button>
  </div>
);
```

Ippo error & loading false → normal card display

User click purchased → only that card update

dummybawaah → toggle button → UI changes if needed

6 Summary (Tanglish)

State → track purchased, dummybawaah, loading, error

useEffect → fetch data once + error handling

if (error) → fetch fail → show UI only one time → return exit

if (loading) → fetch in progress → show UI only one time → return exit

Normal UI → fetch success → card display, buttons work

Child state → purchased / dummybawaah only component-specific

Example code

```
import { useEffect, useState } from "react";

function New(props) {

  // ✅ Component state

  const [purchased, setPurchased] = useState(false); // track
purchase

  const [dummybawaah, setdummybawaah] = useState(true); // temp
toggle

  const [error, setError] = useState(null); // fetch error

  const [loading, setLoading] = useState(true); // fetch loading

  // ✅ Fetch data on first render

  useEffect(() => {

    const timer = setTimeout(() => {

      fetch("http://localhost:3000/elements") // API call
```

```
.then(response => {  
    if (!response.ok) throw new Error("Could not retrieve  
data"); // HTTP error  
  
    return response.json();  
  
})  
  
.then(data => {  
    console.log("Fetched data:", data);  
    setLoading(false); // fetch success  
  
})  
  
.catch(err => {  
    console.log("Fetch error:", err.message);  
    setError(err.message); // store error  
    setLoading(false);  
  
});  
  
    }, 1000); // simulate network delay  
  
}, []);  
  
// ✅ Handle purchase button click  
  
function calender() {  
    console.log(props.month + " monthly calendar purchased");  
    setPurchased(true); // update purchased state  
}  
  
// ✅ Debug logs for state changes  
  
useEffect(() => { console.log(purchased); }, [purchased]);  
  
useEffect(() => { console.log(dummybawaah); }, [dummybawaah]);
```

```
// ✅ Conditional rendering

if (error) return <p>{error}</p>; // show error only once

if (loading) return <p>Loading...</p>; // show loading only once

// ✅ Normal component UI after fetch success

return (

  <div className="card">

    <img src={props.img} alt={props.month} />

    <h1>{props.month}</h1>

    <p className="txt">{props.desc}</p>

    <h2>{props.no}</h2>

    <button onClick={calender}>

      Click here to buy free monthly calendar

    </button>

    <p>{purchased ? "Already getted" : "Get it now"}</p>

    <button onClick={() => setdummybawaah(false)}>Click
man</button>

  </div>

);

}

export default New;
```

FallBack In React

"Fallback" means **backup** – something that kicks in automatically when the main thing fails. In React or frontend code, we use fallback to make sure the UI doesn't break even if something goes wrong

✓ Common Fallback Examples

1. Image fallback

If the image fails to load, show a backup:

```
<img
  src={props.img}
  alt="month"
  onError={(e) => (e.target.src = "fallback.png")}
/>
```

🧠 "Image missing aagumbothu fallback.png kaattanum."

2. Text fallback

If a prop is missing, show default text:

```
<h1>{props.month || "Unknown Month"}</h1>
```

🧠 "Month illa-na 'Unknown Month' nu kaattanum."

3. Component fallback

If data is loading or error occurs:

```
if (error) return <p>Error loading data</p>;
```

```
if (loading) return <p>Loading...</p>;
```

🧠 "Server response varala-na fallback message kaattanum."

✓ Why fallback is important

Situation	Without fallback	With fallback
Image	fails Broken image	Backup image

Data	missing Blank screen	Default text
Fetch	error Crash or freeze	Error message

14 . What is a Custom Hook?

A **custom hook** is just a **reusable function** that uses React hooks (`useState`, `useEffect`, etc.) inside it. It lets you **extract logic** from components and **reuse it** across multiple places.

"Custom hook na oru function maari thaaan. But it uses React hooks inside. Namma arguments pass pannalaam, and it will return values like `[data, error, loading]` – reusable logic, reusable return."

```

USEFETCH.JSX

import { useEffect, useState } from "react";

const useFetch = (url) => {

  const [purchased, setPurchased] = useState(null);           // ✓ holds
  fetched data

  const [error, setError] = useState(null);                   // ✓ holds
  error message

  const [loading, setLoading] = useState(true);               // ✓ loading
  state

  useEffect(() => {

    const timer = setTimeout(() => {

      fetch(url)

        .then(response => {

          if (!response.ok) {

            throw new Error("Could not retrieve data");

          }

        })

    }, 1000);

  });

```

```

        return response.json();
    })

    .then(data => {

        console.log("Fetched data:", data);

        setPurchased(data); // ✅ store in purchased

        setLoading(false);

    })

    .catch(err => {

        console.log("Fetch error:", err.message);

        setError(err.message);

        setLoading(false);

    });

}, 1000);

return () => clearTimeout(timer); // ✅ cleanup

}, [url]);

return [purchased, setPurchased, error, loading]; // ✅ array
return

};

export default useFetch;

```

```

NEW.JSX

import { useEffect, useState } from "react";

import useFetch from "../useFetch"; // ✅ correct import

function New(props) {

```

```

    const [dummybawaah, setdummybawaah] = useState(true); //  local
toggle

    const [purchased, setPurchased, error, loading] =
useFetch("http://localhost:3000/elements"); //  hook usage

    function calender() {

        console.log(props.month + " monthly calendar has been
purchased");

        setPurchased(true); //  updates state from hook
    }

    useEffect(() => {

        console.log("Purchased:", purchased);

    }, [purchased]);

    useEffect(() => {

        console.log("Dummybawaah:", dummybawaah);

    }, [dummybawaah]);

    //  Conditional rendering

    if (error) return <p>{error}</p>;

    if (loading) return <p>Loading...</p>;

    return (

        <div className="card">

            <img src={props.img} alt="month" />

            <h1>{props.month}</h1>

            <p className="txt">{props.desc}</p>

            <h2>{props.no}</h2>

```

```

    <button onClick={calender}>Click here to buy free monthly
calendar</button>

    <p>{purchased === true ? "Already getted" : "Get it now"}</p>

    <button onClick={() => setdummybawaah(false)}>Click
man</button>

  </div>

);
}

export default New;

```

useFetch.jsx

- Custom hook that fetches data from a URL.
- Uses `useState` for data (`purchased`), `error`, and `loading`.
- Runs once when URL changes (`useEffect`).
- Waits 1 sec, fetches data → if success, saves it in `purchased`; if fails, sets `error`.
- Returns `[purchased, setPurchased, error, loading]`.

New.jsx

- Imports the hook and uses it to get data.
- Has its own state `dummybawaah` for testing.
- `calender()` updates `purchased` to `true` and logs month info.
- Shows **Loading...**, **Error**, or fetched data conditionally.
- Buttons used to trigger actions and update states.

15. What is `onChange` in React?

onChange in React?

`onChange` is an event attribute (like `onClick`, `onMouseOver`, etc.)

👉 It is used to detect changes in input fields – not directly to compare, but you can use it to compare values while user types.

⚡ Step 1: Basic idea of onChange

In React, whenever user types in an `<input>`, the `onChange` event triggers. We can use this event to update state and then compare values.

🧠 Step 2: Example – Compare Two Inputs

Let's say we have two text boxes, and we want to show:

- ✅ "Match" if both are same
- ❌ "Not matching" if different

```
CompareInputs.jsx

import { useState } from "react";

function CompareInputs() {

  const [input1, setInput1] = useState("");

  const [input2, setInput2] = useState("");

  const [message, setMessage] = useState("");

  // ♦ Function for first input

  function handleChange1(e) {

    setInput1(e.target.value);

  }

  // ♦ Function for second input + comparison
```

```
function handleChange2(e) {

  const value = e.target.value;

  setInput2(value);

  if (input1 === value) {

    setMessage("✅ Both inputs match!");

  } else {

    setMessage("❌ Inputs are different!");

  }

}

return (

  <div style={{ textAlign: "center", marginTop: "50px" }}>

    <input

      type="text"

      placeholder="Enter first text"

      value={input1}

      onChange={handleChange1}

    />

    <br /><br />

    <input

      type="text"

      placeholder="Enter second text"

      value={input2}

      onChange={handleChange2}


```

```
    />

    <br /><br />

    <p>{message}</p>

  </div>

);  }

export default CompareInputs;
```

The Logic Of The Code :

- `input1`, `input2`, and `message` are **states**.
 - `handleChange1` & `handleChange2` are **functions** to update those states.
 - The `if-else` compares both inputs and updates `message`.
 - Inside `return`, the inputs are displayed and linked to their handlers.
-

16. What is React Router?

React Router is a **library** that allows you to create **multiple pages (routes)** inside a **Single Page Application (SPA)**.

Instead of reloading the entire website, React Router **switches components** dynamically.

✓ Example:

- `/` → Home page component
 - `/about` → About page component
 - `/contact` → Contact page component
-

⚙️ 2 Installing React Router

First, install it in your project folder (only once):

```
npm install react-router-dom
```

Then import it in your `App.jsx`.

```
main.jsx

import { StrictMode } from "react";

import { createRoot } from "react-dom/client";

import "./index.css";

import Newlist from './Newlist.jsx';

import Counter from './counter.jsx';

import Front from './Front.jsx';

import Compare from './Compare.jsx';

import { createBrowserRouter, RouterProvider } from
"react-router-dom";

const route = createBrowserRouter([

  {

    path: "/", //  default route

    element: <Front />

  },

  {

    path: "/newlist",

    element: <Newlist />

  },

  {

    path: "/counter",
```

```

    element: <Counter cli={ (name) => alert("Hi " + name + ", welcome
to learning React")} />

  },

  {

    path: "/compare",

    element: <Compare />

  }

]);

```

```

createRoot(document.getElementById("root")).render(

```

```

  <StrictMode>

```

```

    <RouterProvider router={route} />

```

```

  </StrictMode>

```

```

);

```



Front.jsx

```

import { Link } from "react-router-dom";

```

```

function Front() {

```

```

  return (

```

```

    <div>

```

```

      <h1>Hii Welcome To Monthly Calendar Purchase Site</h1>

```

```

      <Link to="/newlist"><button>Go to Calendar List</button></Link>

```

```

      <Link to="/compare"><button>Compare Inputs</button></Link>

```

```

      <Link to="/counter"><button>Counter</button></Link>

```

```

    </div>

```

```
);
}

export default Front;
```

How it Works :-

- <BrowserRouter>** - Enables Routing in your React app
- <Routes>** - Container that holds all your routes
- <Route>** - Defines path (URL) & ^{which} component to render
- <Link>** - Used for Navigation without reloading page

<Link> & <a> tag :-

<a>	<Link>
Refreshes the entire page (page reload)	Changes component without reload (no page reload)
Breaks single-app flow (react friendly no)	Keeps React app fast & smooth (React friendly yes)
eg:- About (Props href = "/about")	eg:- <Link to="/about">About</Link> (Props to = "/about")
Normal website/external	React Single page app nav

Example flow :-

- When user clicks <Link to="/about"> -> URL changes to /about/
- React Router renders about.jsx
- No full page reload -> Smooth transitions

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <Home />,
    errorElement: <Error />
  },
  {
    path: '/login',
    element: <Login />,
    errorElement: <Error />
  }
]);
```

Install the proper react router version.
Throwing a Manual Error.

```
import { Link } from 'react-router-dom';

function Error() {
  return (
    <div>
      <p>Server busy</p>
    </div>
  );
}
```

import default Error

Link :
Link is a component in React.

Import {Link} from "react-router-dom";

Diff <a> tag and <Link> Component :

- <a> tag -> Sending to server and gets in
- <Link> Component -> The browser router will handle

Reason (As props)

<Link className="btn btn-primary" to="/">Home </Link>

Component (Navigation) | className (styling) | to (path to go) | element (text shown in link) | errorElement (closing tag)

Router => It defines routes (Shows for which url) (Defines which comp)

Render => show route (Showing the matchable comp to) (Display component on page) (url)

Navigate => Moves page without reload. (move to another page without reloading)

17. What is Cleanup in React?

Cleanup in React?

When a component **unmounts** (like your Home page when user clicks Shirt), React will **remove the UI**, but **any background tasks** like `setInterval`, `setTimeout`, or even **event listeners** will **keep running** unless you clean them up.

🔥 Why cleanup is important:

- Prevents **memory leaks**
- Avoids **unwanted background tasks**
- Stops **server or DB calls** from old components
- Keeps your app **fast and efficient**

```
useEffect(() => {  
  
  const interval = setInterval(() => {  
  
    console.log("Home page polling server...");  
  
  }, 1000);  
  
  return () => {  
  
    clearInterval(interval); // ✅ cleanup when Home unmounts  
  
    console.log("Home page cleanup done");  
  
  };  
  
}, []);
```

✅ What Happens Without Cleanup :

- .)Even after Home page unmounts, the interval keeps running
- .)It may keep calling server or DB
- .)It may cause duplicate fetches, slow UI, or crashes

Update function :

An update function lets you to update a state depend on its previous value instead of directly passing a newly value.

Purpose:-

- ① Helps when new state depends on old state value
- ② Prevents state or outdated updates
- ③ Works well with react batch optimization

Example:-

```
function Count () {
  const [count, setCount] = useState(0);

  function inc () {
    setCount(count + 1);
    console.log(count);

    // setCount(prevCount => prevCount + 1);
    // setCount(prevCount => prevCount + 1);
  }
}
```

```
useEffect(() => {
  console.log(count);
}, [count]);

return (
  <>
    <p> {count} </p>
    <button onClick={inc}> inc val </button>
  </>
)

before default count;
```

Batch Optimization (1) Cleanup Error.

① :- `const [count, setCount] = useState(0);`

- React gets mul state updates → re-renders only once → faster UI.
- Problem: Using `setstate(Oldval + 1)` multiple times → Uses ~~old~~ value → wrong result.
- Fix: Use update function.

```
setCount(prevCount => prevCount + 1);
```

∴ (Error you see: stale state).

18. Usecontext in react

useContext — "Shared data between components"

Concept:

`useContext` allows you to **share data** (like username, theme, or login info) between multiple components **without passing props manually** at every level.

Steps:

1. Create Context
2. Wrap parent with Provider
3. Access data using useContext

Example:

```
import { createContext, useContext } from "react";

// 1. Context create panrom

const UserContext = createContext();

function App() {

  const username = "Dinesh Kumar";

  return (

    // 2. Provider la value kudukrom

    <UserContext.Provider value={username}>

      <Child />

    </UserContext.Provider>

  );
}

function Child() {

  // 3. useContext la access panrom

  const name = useContext(UserContext);

  return <h2>Welcome, {name} 🙌</h2>;
}

export default App;
```

19. useRef in react

useRef :-

Holds a Mutable (Changeable) Value That Persist across renders. Does n't re-render when updated. Alternative to useState for values that change but don't need to display immediately.

Syntax :-

Import { useRef } from "react";

const refVar = useRef(initialValue);

Access Value :- refVar.current

Updated Value :- refVar.current = New Value

Example :-

Import { useRef, useState } from "react";

function Count() {

const [Count, SetCount] = useState(0);

const refCount = useRef(0);

function Inc1() {

refCount.current += 1;

console.log("refCnt", refCount.current);

}

function Inc2() {

refCount.current

SetCount(count + 1);

}

return (

<>

<p> { Count } </p>

<button onClick={inc1}> Inc state </button>

<p> { refCount.current }

<button onClick={inc2}> Inc ref </button>

</>

};

Key-Points :-

·) UI does not auto-update with useRef.

·) Good for

·) Timers (setInterval, Timeout)

·) Previous state tracking

·) DOM interference.

·) To show ref val in UI (You must trigger a re-render (Use useState or forceUpdate))

useRef vs useState

Feature	useRef	useState
Updates UI	No	Yes
Triggers re-renders	No	Yes
Example Usage	Timer, Prev-Value	Counter, Input Value

useRef = Internal mutable storage, does not affect UI automatically, perfect for keeping data without rendering.

20. Dynamic Path

Dynamic Path :-
 Dynamic Path Means Route Without Variable Part. Example: /Calendar/:month (/or) /Product/:id

How to Define a Dynamic Route

```

    {
      path: "/Calendar/:month",
      element: <Calendar />
    }
  
```

∴ month is the dynamic part.
 ∴ You can access it using useparams().

How to Read dynamic Value :

```

    import { useparams } from "react-router-dom";
    const { month } = useparams();
  
```

Dynamic Path means a route that changes based on Variable data (like ID or Name).

Example Code :-

Main.js :-

```

import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { createBrowserRouter, RouterProvider } from "react-router-dom";
import App from "./App";
import User from "./User";

const router = createBrowserRouter([
  { path: "/", element: <App /> },
  { path: "/users/:id", element: <User /> }, // dynamic path
]);

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <RouterProvider router={router} />
  </StrictMode>
);
  
```

User.js :-

```

import { useparams } from "react-router-dom";

function User() {
  const { id } = useparams();
  // useparams gets the value from url.
}
  
```

```

return (
  <div>
    <h1> User ID: {id} </h1>
  </div>
);
}
Export default User;

```

App.js:

```

import {Link} from "react-router-dom";
function App() {
  return (
    <div>
      <h1> Home page </h1>
      <Link to="/user/1"> User 1 </Link>
      <Link to="/user/2"> User 2 </Link>
    </div>
  );
}
Export default App;

```

.) id → is a route parameter. (dynamic part of url)

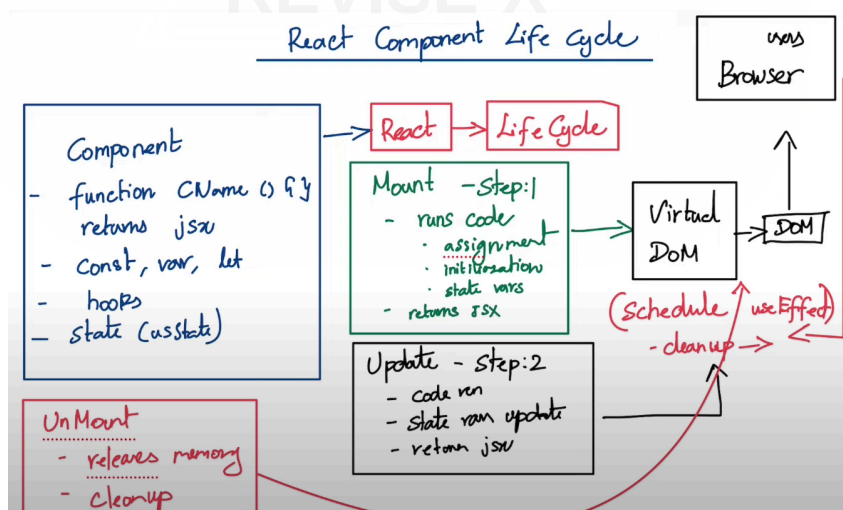
.) Use `params()` → Hook used to get the parameter value in that route.

.) Used for → Profiles, Products, details, Pages.

.) Example in real apps :-

- 1 Product / 10
- 1 User / dinesh
- 1 Post / abc123

21. Life Cycle React Component



React Component Life Cycle – Full Explanation (Tanglish + Code Examples)

Component

Meaning:

Component-na oru **function** or **class** that returns **JSX (UI structure)**.
React la ellam components-a thaam build pannuvom.

Inside a Component:

- Function component (normal method)
- Variables (const, let, var)
- Hooks (like `useState`, `useEffect`)
- State variables (UI-la reflect aagura data)

Example:

```
import { useState } from "react";

function Counter() {

  // state variable

  const [count, setCount] = useState(0);


  // returns JSX

  return (

    <div>
```

```
    <h2>Count: {count}</h2>

    <button onClick={() => setCount(count + 1)}>Increment</button>

  </div>

  );
}

export default Counter;
```

Tanglish Explanation:

👉 Ithu oru functional component.
`useState` la irukkura value (`count`) change aana odane React UI update pannum.
JSX return pannura function-na adhu React component.

Mount (Step 1)

💡 Meaning:

Mount-na component first time screen-la **render** aagura time.
React component-a virtual DOM la convert pannitu, browser-la DOM-a update pannum.

⚙️ Steps inside Mount:

1. Code first time run aagum (assignments, initialization, state vars).
2. JSX return aagum → Virtual DOM create aagum.
3. Virtual DOM → Real DOM (screen-la UI kaanum).
4. `useEffect(() => {}, [])` effect run aagum.

🧠 Example:

```
import { useEffect } from "react";
```

```
function Welcome() {  
  useEffect(() => {  
    console.log("Component mounted"); // Mount time la run aagum  
  }, []); // empty deps => runs only once  
  
  return <h1>Welcome User</h1>;  
}
```

Tanglish Explanation:

👉 Component first render aana podhu mount phase start aagum.
useEffect la empty array kudutha, adhu mount time la oru thadava mattum run aagum.
JSX virtual DOM la create pannitu browser DOM-ku render pannum.

 Virtual DOM → DOM **Meaning:**

React oda *Virtual DOM*-na browser DOM oda **copy**, memory la store aagura oru light version.

 Process:

1. Component JSX → Virtual DOM object form.
2. React compare (diff) new VDOM with old VDOM.
3. Only changed parts (diffs) update pannum in real DOM.
(Full UI-a re-draw panna maata.)

 Example:

```
function Message({ text }) {
```

```
    return <p>{text}</p>;  
  }
```

If prop `text` changes from "Hello" → "Hi",
React only updates that `<p>` element's text, full page re-render aagala.

Tanglish Explanation:

👉 Virtual DOM-na React oda duplicate browser UI memory copy.
React rendu virtual DOM compare panni only changed part-a real DOM-la update pannum.
Adhanal React speed-ah irukku.

Update (Step 2)

💡 Meaning:

Update phase-na state or props change aana time React re-render aagum.

⚙️ Steps:

1. Component re-run aagum (state/props change na).
2. JSX → new Virtual DOM create aagum.
3. React diff panni update pannum real DOM.
4. `useEffect` with dependency array `[value]` run aagum.

🧠 Example:

```
import { useState, useEffect } from "react";
```

```
function Counter() {  
  
  const [num, setNum] = useState(0);
```

```

useEffect(() => {

  console.log("Updated value:", num);

}, [num]); // Runs when num changes

return (

  <>

    <p>Number: {num}</p>

    <button onClick={() => setNum(num + 1)}>Add</button>

  </>

);
}

```

Tanglish Explanation:

👉 **setNum** use pannina odane React component update aagum.
 New virtual DOM create pannu old-oda compare pannum.
num value change aana odane **useEffect** run aagum (update effect).

🌀 **useEffect & Cleanup (Arrow Section)**

💡 **Meaning:**

useEffect side effects handle panna use pannuvom (like API call, timer, event listener, etc.)

cleanup-na remove panna code (component close aagumbodhu or effect re-run aagumbodhu).

🧠 **Example:**

```
import { useEffect } from "react";
```

```
function Timer() {
```

```
useEffect(() => {

  const timer = setInterval(() => console.log("Running..."), 1000);

  // cleanup

  return () => {

    clearInterval(timer);

    console.log("Timer cleared");

  };

}, []);

return <p>Timer Started</p>;

}
```

Tanglish Explanation:

👉 Ithu mount time la run aagum, unmount aana bodhu cleanup run aagum. **useEffect** return function la kodukkura code-na cleanup section. Timer, event listener, subscription ellam remove pannum unmount la.

Unmount

💡 Meaning:

Unmount phase-na component screen-la irundhu remove aagura time.

⚙️ Steps:

1. React DOM-la irundhu element remove pannum.
2. Cleanup (memory release) nadakkum.
3. **useEffect** return function run aagum.

Example:

```
import { useState } from "react";

import Child from "./Child";

function Parent() {

  const [show, setShow] = useState(true);

  return (

    <>

      <button onClick={() => setShow(!show)}>Toggle</button>

      {show && <Child />}

    </>

  );
}

function Child() {

  useEffect(() => {

    console.log("Child Mounted");

    return () => console.log("Child Unmounted");

  }, []);

  return <h2>Child Component</h2>;
}
```

Tanglish Explanation:

👉 Button click panni **show** false aagumbodhu, Child component remove aagum (unmount).

Unmount aana time la cleanup effect run aagum – memory, timer, event ellam clear pannum.



Full Lifecycle Summary (Easy Table Form)

Phase	When Happens	Hook/Code Used	Tanglish Meaning
Mount	First render	<code>useEffect(() => {}, [])</code>	First time component load aagum
Update	State/props change	<code>useEffect(() => {}, [deps])</code>	UI re-render aagum
Unmount	Component remove	Cleanup return	Memory clear aagum



Extra Concepts From Diagram



Schedule → useEffect

React schedule pannum – means useEffect run pannum DOM update apram. Adhu render blocking illama irukka help pannum.



Cleanup

Cleanup run pannum time:

- Before new effect run (dep change)
- At component unmount.



Browser box

Virtual DOM → Real DOM → browser-la show aagum.

Tanglish Summary Paragraph

React component oda life cycle na 3 phase – **Mount, Update, Unmount**.

Mount la component first time UI la varum; state initialize pannum; JSX virtual DOM la create pannitu browser DOM ku render pannum.

Update la state or props change aana odane component re-render aagum; React diff panini only changed part update pannum.

Unmount la component remove aagum; cleanup nadakkum; timers, event listeners, memory leak avoid aagum.

useEffect side effects handle panna use pannuvom; adhu return pannura function cleanup pannum.

Small Practical Demo Flow

```
function Demo() {  
  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
    console.log("Mounted");  
  
    return () => console.log("Unmounted");  
  }, []);  
  
  useEffect(() => {  
    console.log("Count updated:", count);  
  }, [count]);  
  
  return (  

```

```

    <>

    <p>Count: {count}</p>

    <button onClick={() => setCount(count + 1)}>Add</button>

  </>

  );
}

```

👉 Output in console:

Mounted

Count updated: 0

Count updated: 1

Count updated: 2

Unmounted

REVISE-X